Тема 22: Проектиране и интегриране на софтуерни системи.

ТЕОРИЯ

1. Характеристики на разпределените системи

Разпределена система = група от независими компютри, свързани в мрежа, които за потребителя изглеждат като **една цялостна система**. Комуникацията е само чрез **изпращане на съобщения**.

Цели: висока производителност, надеждност, мащабируемост, лесна поддръжка.

Видове разпределени системи

Вид	Описание	Примери
Разпределени изчислителни системи	За задачи с висока производителност	Клъстер, Грид
Разпределени информационни системи	Свързване на приложения в мрежа	Client-Server, P2P
Широкоразпространени	Голям географски обхват	Интернет, IoT

Разпределени изчислителни системи

Тип	Хардуер	ОС	Физическо местоположение
Клъстер	Идентичен	Еднаква	Близко, 1 мрежа
Грид	Хетерогенен	Различни	Отдалечени

Тенденции

- Широкоразпространени мрежови технологии
- Мобилни и IoT устройства в разпределени системи
- Интернет = голяма разпределена система
- Нарастване на мултимедийни и on-demand услуги
- **Cloud computing**: AWS, Azure, Google Cloud

---

2. Междупроцесна комуникация (IPC)

IPC (Inter-Process Communication) = обмен на съобщения между процеси.

ОС предоставя механизми: **message queue**, **семафори**, **shared memory**.

> *На практика се използва middleware като обвивка, предоставяща по-висока абстракция.*

Два вида:

- **Connection-oriented** — изграждане на канал специално за комуникацията
- **Message-oriented** — преизползване на съществуващ канал

2.1 Отдалечено извикване — RPC и RMI

RPC (Remote Procedure Call)

Позволява на процес B да извика процедура на процес A **прозрачно** (като локална).

- Интерфейсите се дефинират чрез **IDL** (Interface Definition Language)
- Скрива реализацията на другия модул

Семантики на изпълнение:

Семантика	Поведение
maybe	Процедурата се изпълнява веднъж или въобще не
at-least-once	Клиентът получава резултат; процедурата е изпълнена поне веднъж
at-most-once	Клиентът получава резултат; процедурата е изпълнена **най-много** веднъж

> ***gRPC*** *— модерна реализация на RPC от Google; бинарен протокол (Protocol Buffers).*

RMI (Remote Method Invocation)

Обектно-ориентиран метод за отдалечено извикване (Java-специфично).

Прилика с RPC: Интерфейси + заявка-отговор + прозрачност.

Разлики от RPC:

- Работи с **обектно-ориентирана парадигма**
- Може да предава **референции към обекти** като параметри

Компоненти на RMI:

Компонент	Роля
Communication Module	Препраща заявки/отговори между клиент и сървър
Remote Reference Module	Съпоставя локални и отдалечени референции; маршализация/демаршализация
Servant	Инстанция на отдалечения обект (на сървъра)
Proxy/Stub	Имплементира отдалечения интерфейс на **клиента**; препраща заявките
Skeleton	Имплементира отдалечения интерфейс на **сървъра**; делегира на Servant
Dispatcher	Получава заявки от communication module; насочва към Skeleton

Поток:

```
Клиент → Proxy/Stub → [Мрежа] → Communication Module → Dispatcher → Skeleton → Servant
```

2.2 Мултикаст (Multicast)

Мултикаст = изпращане на съобщение до **група от получатели** едновременно.

Употреба: Стрийминг на видео, онлайн игри, разпределени БД.

Gossip-Based Data Dissemination (Алчен/Епидемичен алгоритъм):

- Всеки **заразен** възел (притежава информация) я предава на съседите си
- Аналогично на разпространение на вирус
- Постига се висока отказоустойчивост и гарантирана доставка

---

3. Разпределени обекти и компоненти

Разпределени обекти

Прилага **ООП** при разработването на разпределени системи. Комуникиращите единици са **обекти**, комуникират чрез **RMI** или **разпределени събития**.

Разпределени компоненти

Компонент = градивна единица с описани:

- Интерфейси, които предлага (**provides interface**)
- Контекстни зависимости (**requires interface**)

Компонентите се свързват чрез **middleware**.

Примери:

- **Enterprise Java Beans (EJB)** — компоненти за Java enterprise приложения
- **CORBA** (Common Object Request Broker Architecture) — платформено-независима система за разпределени обекти; ORB (Object Request Broker) управлява комуникацията

---

4. Уеб услуги

Уеб услуга = софтуерен компонент, предоставящ функционалност чрез **стандартизирани интернет протоколи** (HTTP).

- Използва **XML или JSON** за обмен на данни
- Достъпна от различни платформи и езици
- Може да използва **REST** архитектура
- **Цел**: Абстрахиране на изпълнението от имплементацията

Шаблони за комуникация

Шаблон	Описание	Блокиране
Синхронна	Клиентът чака отговор	**Блокира** се
Асинхронна	Клиентът не чака	Не се блокира
Базирана на събития	Съобщения при настъпване на събитие (webhooks)	Не се блокира

4.1 SOAP (Simple Object Access Protocol)

SOAP = протокол за размяна на XML съобщения, имплементиран над HTTP или SMTP.

Поддържа: Синхронна + асинхронна комуникация; разширяемост; сигурност; надеждност.

Употреба: Сложни и критични бизнес приложения.

SOAP дефинира:

- Структурата на XML съобщенията
- Схема заявка-отговор
- Схема за обмен на документи
- Правила за обработка на XML елементи

Структура на SOAP съобщение:

```
<Envelope>
  <Header>
    <!-- допълнителна информация: сигурност, маршрутизация -->
  </Header>
  <Body>
    <!-- основните данни: заявка или отговор -->
    <getProductRequest>
      <productId>12345</productId>
    </getProductRequest>
  </Body>
</Envelope>
```

4.2 WSDL (Web Services Description Language)

WSDL = XML-базиран език за **описание** на уеб услуги.

WSDL дефинира:

- Интерфейса на услугата (методи, параметри, типове данни)
- Начина на комуникация

Използва се като **допълнение към SOAP** — WSDL описва интерфейса, SOAP го имплементира.

```
<definitions name="ProductService">
  <message name="getProductRequest">
    <part name="productId" type="xsd:string"/>
  </message>
  <portType name="ProductPortType">
    <operation name="getProduct">
      <input message="getProductRequest"/>
      <output message="getProductResponse"/>
    </operation>
  </portType>
</definitions>
```

4.3 UDDI (Universal Description, Discovery and Integration)

UDDI = стандарт за **публикуване и откриване** на уеб услуги. Като **телефонен указател** за услуги.

Доставчиците регистрират услугите си; клиентите ги намират.

Три типа информация:

Тип	Съдържание

Бели страници	Информация за **доставчика** (компания, контакти)
Жълти страници	**Класификация** на услугите по категории
Зелени страници	**Техническа** информация (WSDL описания)

REST vs SOAP

Критерий	REST	SOAP
Протокол	HTTP	HTTP, SMTP
Формат	JSON, XML	XML
Стандарти	Неформални	SOAP, WSDL, UDDI
Сложност	По-прост	По-сложен
Сигурност	HTTPS + токени	WS-Security
Употреба	Уеб API, мобилни	Enterprise, банков
Операции	GET/POST/PUT/DELETE	Методи по имена

Връзката между стандартите

```
┌─────────────────────────────────────────────┐
│                  Уеб услуга                 │
│                                             │
│  WSDL ——— описва интерфейса                 │
│  SOAP ——— осъществява комуникацията         │
│  UDDI ——— публикува и открива услугите      │
└─────────────────────────────────────────────┘

Клиент → (1) Търси в UDDI → (2) Получава WSDL описание →
       → (3) Изпраща SOAP заявка → Сървър
```

Бърза справка

Концепция	Ключ
Клъстер	Хомогенни машини, близко, 1 ОС
Грид	Хетерогенни, отдалечени, различни ОС
RPC	Извикване на отдалечена процедура; IDL; maybe/at-least/at-most
RMI	ООП RPC; Proxy+Skeleton+Servant
Gossip	Епидемичен алгоритъм за разпространение
Мултикаст	Съобщение до група получатели

CORBA	Платформено-независими разпределени обекти; ORB
SOAP	XML съобщения над HTTP; Envelope+Header+Body
WSDL	XML описание на услуга (интерфейс, методи)
UDDI	Регистър за услуги; бели/жълти/зелени страници
Синхронна	Клиентът блокира и чака отговор
Асинхронна	Клиентът не блокира